

Empathy Map Canvas

Date	1 October 2026	Team ID / Name	SWTID-2026-6919
Project Name	Script-Controlled ACL	Maximum Marks	4 Marks

1. What is an Empathy Map Canvas?

An empathy map is a simple, easy-to-digest visual that captures knowledge about a user's behaviors, attitudes, and emotional drivers. It is a powerful human-centered design tool that helps project teams gain a deep, shared understanding of their target users. Creating an effective solution requires understanding the true operational problem and the person experiencing it. The exercise of creating this map helps team members step into the shoes of ServiceNow Administrators, Security Engineers, Process Owners, and Support Fulfillers—evaluating their goals, frustrations, and environment.

Reference Template:<https://www.mural.co/templates/empathy-map-canvas>

2. Target User Persona Profile

- **Primary Persona:** ServiceNow System Administrator / Security Engineer & Department Manager
- **Core Responsibility:** Configures record-level access security (ACLs), manages field permissions based on dynamic values (State, Department, Confidentiality), and ensures seamless fulfiller user experience.
- **Key Objective:** Enforce dynamic field-level security without causing list truncation warnings or exposing confidential record details across global search and reporting.

3. Empathy Map Canvas Matrix

THINK AND FEEL?

Whatreallycounts | Preoccupations | Worries & Aspirations

- How can I dynamically restrict record access based on specific field values (e.g., State, Department)?
- Static role-based ACLs are too broad; I worry about sensitive record exposure during audits.
- I want security rules to execute server-side seamlessly without breaking global search or reporting widgets.
- Concerned that poorly optimized ACL scripts might degrade platform database query performance.

SEE?

Environment | System State | Market & Platform Behavior

- Out-of-the-box condition builders that cannot compare session attributes (gs.getUser()) against record fields dynamically.
- Users seeing 'Security constraints prevent viewing' warnings on list views when row read ACLs block records.
- Confidential tickets appearing in global search summaries or report widgets when role checks fail.
- Multiple active roles assigned to users that inadvertently bypass static access controls.

HEAR?

StakeholderInputs | Manager Demands | Influencers & Audits

- **Compliance & Audit:** "Enforce strict row and field-level privacy based on active record states!"
- **Fulfillers & End Users:** "Why am I seeing security truncation errors?" or "Why can't I edit this assigned incident?"
- **ServiceNow Architects:** "Never perform heavy GlideRecord loops inside scripted ACLs; pair them with Before Query Business Rules!"

SAY AND DO?

PublicAttitude | System Behavior | Implementation Actions

- Writes server-side ACL script logic (setting answer = true / false) using gs.getUserID() and record field evaluations.
- Configures 'Before Query' Business Rules to filter restricted records at the database query level.
- Uses 'Impersonate User' to rigorously test permissions across different user roles and departments.
- Maintains clear security documentation and field-level access matrices for compliance audits.

PAIN (FEARS & FRUSTRATIONS)

Barriers | SecurityGaps | OperationalObstacles

- Frustration with static out-of-the-box ACL condition limits.
- Anxiety over data breaches, privacy leaks, or audit non-compliance.
- User friction caused by unhandled security warnings on list views.
- Fear of creating complex scripts that are hard to maintain.

GAIN (WANTS & SUCCESS CRITERIA)

DesiredOutcomes | MeasuresofSuccess | ValueDelivered

- Granular, field-value-based dynamic access control.
- Seamless user experience without cryptic list view security warnings.
- Full compliance with corporate data privacy & security standards.
- Clean, maintainable, high-performance server-side security logic.

4. Detailed Analysis of User Perspectives

4.1 Think & Feel (User Mindset & Emotional State)

- **Core Preoccupation:** Ensuring that sensitive records (e.g. Confidential or Closed status) are dynamically protected from unauthorized viewing or modification based on specific field values rather than static roles.
- **Security Worry:** Unprotected field-level access creating compliance vulnerabilities during security audits or allowing sensitive departmental data to be queried in reports.
- **Aspiration:** Implementing an automated, script-controlled ACL framework that dynamically evaluates `current.field_name` against session properties like `gs.getUser()` cleanly.

4.2 Hear (External Influences & Feedback)

- **Management & Compliance:** Strong directive to enforce zero-trust field-level data privacy across department boundaries.
- **End Users & Support Staff:** Express confusion when encountering 'Security constraints prevent viewing' notifications or empty list rows.
- **System Architects:** Advise keeping ACL scripts lightweight and implementing 'Before Query' business rules to handle row-level filtering silently at the database layer.

4.3 See (Current System Environment)

- **Out-Of-The-Box Limitations:** Standard ServiceNow ACL condition builders can only evaluate simple, static field equality and static user role checks.
- **User Interface Signs:** Fulfillers navigating list views see security truncation notices when row-level read ACLs evaluate to false.
- **Reporting & Search Exposure:** Without scripted access controls, confidential records appear in search indexes and dashboard reporting widgets.

4.4 Say & Do (Observed Actions & Technical Practices)

- **ACL Scripting:** Enables Advanced checkbox on ACL definition, writing JavaScript evaluating `if (current.u_confidential == true && !gs.hasRole('admin')) answer = false;`
- **Before Query BR Alignment:** Creates a 'Before Query' Business Rule on the same table to match ACL conditions, ensuring query results naturally exclude restricted records.
- **Testing & Validation:** Leverages ServiceNow's 'Impersonate User' feature to verify permissions across varied roles, departments, and record field values.

4.5 Pains & Obstacles

- **Static ACL Constraints:** Inability to perform dynamic field-value matching with out-of-the-box role checks.
- **User Friction:** List truncation notices causing end-user confusion and support ticket clutter.
- **Audit Risk:** Potential data exposure due to missing field-level read/write scripted guards.

4.6 Gains & Value Delivered

- **Dynamic Security:** Robust, field-value-driven access control for `read`, `write`, and `create` operations.
- **Seamless UX:** Invisible query-level row filtering without security constraint warning banners.
- **Audit Readiness:** Full compliance with enterprise security requirements and complete audit traceability.